

BharatLens

"See the world through India's lens."

An India-focused geopolitical intelligence platform that transforms global events into source-backed explanations of their potential impact on India.

Complete Project Guide

Architecture | Domain Model | API | AI Pipeline | Development

Version: M15 (AI Evaluation)

Generated: August 2026

Table of Contents

1. Project Overview	
2. Product Vision & Core Loop	
3. Technology Stack	
4. Architecture Overview	
5. Repository Structure	
6. Database Schema & Domain Model	
7. API Reference	
8. Data Ingestion Pipeline	
9. Deduplication Strategy	
10. Claims & Evidence System	
11. AI Analysis Pipeline	
12. Background Processing (BullMQ)	
13. Admin Console & Human Review	
14. Impact Chains	
15. Frontend Pages	
16. Search System	
17. Analytics & Map	
18. Development Setup	
19. Key Commands Reference	
20. Architecture Decision Records	
21. Failure Modes & Observability	
22. AI Evaluation Harness	
23. Design Principles	

1. Project Overview

What BharatLens Does

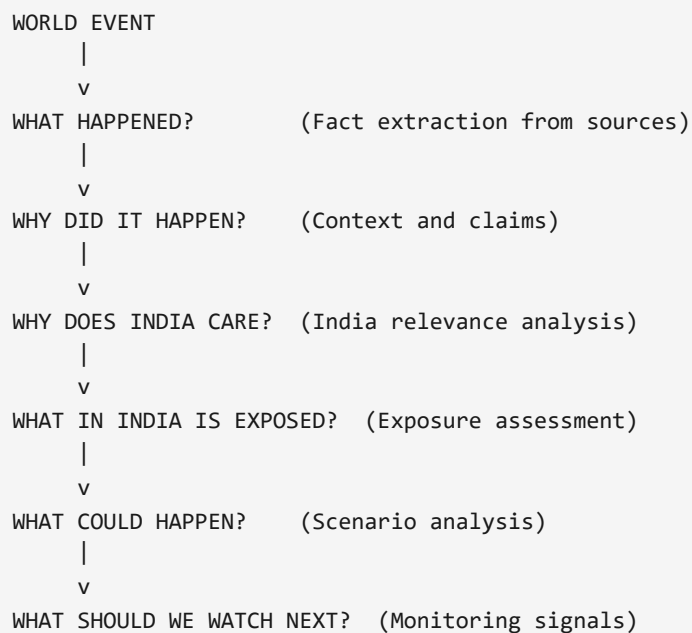
BharatLens answers one core question:

"Something happened in the world. What does it mean for India?"

It is NOT a news aggregator. It is a geopolitical intelligence platform that:

- Ingests news from global sources (RSS feeds)
- Extracts factual claims and evidence
- Classifies events by India-relevant impact categories
- Generates structured impact assessments with source attribution
- Maintains audit trails for AI-generated analysis
- Supports human review before publication

Core Product Loop



Milestone Status

The project is at **M15 (AI Evaluation)**. All milestones M0 through M15 are implemented:

Milestone	Name	Status
M0	Foundation	Done
M1	Core Domain	Done
M2	REST API	Done
M3	Frontend	Done
M4	Event Timeline	Done
M5	News Ingestion	Done
M6	Deduplication	Done
M7	Evidence + Claims	Done
M8	AI Analysis	Done
M9	Background Processing	Done
M10	Admin + Human Review	Done
M11	Impact Chains	Done
M12	Search	Done
M13	Geopolitical Map	Done
M14	Advanced Analytics	Done
M15	AI Evaluation	Done

2. Product Vision & Core Loop

Event Lifecycle

```

SOURCE (RSS feeds, government statements)
|
v
ARTICLE (metadata: title, URL, summary, hash)
|
v
CANDIDATE EVENT (auto-grouped unmatched coverage)
|
v
DEDUPLICATION (URL, content hash, title, entity overlap)
|
v
EVENT (one table, status field distinguishes candidates vs published)
|
+---> Claims --> Evidence
+---> Updates (timeline)
+---> Impact Assessments (v1, v2, v3...)
+---> Impact Chains (v1, v2...)
|
v
HUMAN REVIEW (admin console)
|
v
PUBLISHED EVENT

```

Fact vs Analysis vs Scenario

Every important claim is classified. This is a critical product requirement.

Type	Meaning	Example
FACT	Directly supported by reliable evidence	"Oil prices increased after the disruption."
ANALYSIS	Reasoned interpretation based on facts	"A prolonged disruption could increase India's import costs."
SCENARIO	A possible future outcome	"If the disruption lasts months, inflationary pressure could grow."
UNKNOWN	Insufficient reliable information	"The exact future duration of the disruption."

India Impact Categories

Category	Description
ENERGY	Oil, gas, coal, renewable energy exposure
TRADE	Import/export relationships, tariffs, supply chains
ECONOMY	GDP, inflation, currency, fiscal impact
SECURITY	Regional stability, terrorism, border security
DEFENCE	Military relationships, arms procurement
DIPLOMACY	Bilateral/multilateral relationships
TECHNOLOGY	Tech supply chains, digital infrastructure
SUPPLY_CHAIN	Critical supply chain dependencies
INDIAN_CITIZENS	Diaspora, students, workers abroad

Impact Levels

Level	Description
LOW	Minimal direct impact on India
MEDIUM	Notable exposure, manageable effects
HIGH	Significant direct impact on Indian interests
CRITICAL	Severe immediate threat to Indian interests

Evidence Strength vs Analysis Confidence

These are TWO DIFFERENT fields. They must never be collapsed into one number.

Field	Question	Owner	Values
evidence_strength	How well is this supported by sources?	Computed from linked evidence	WEAK / MODERATE / STRONG
analysis_confidence	How sure is this interpretation?	Reviewer or model (labeled as estimate)	LOW / MEDIUM / HIGH

3. Technology Stack

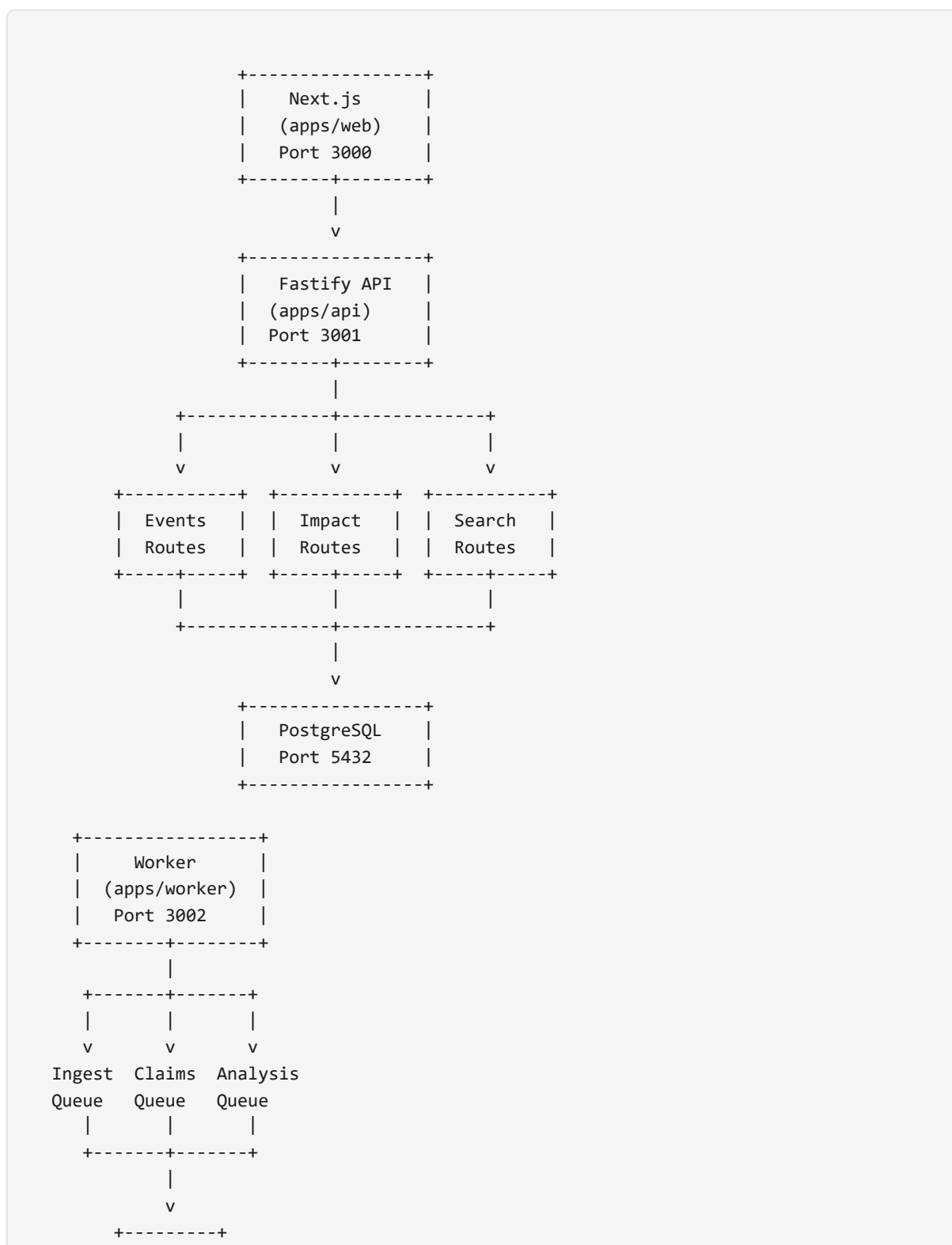
Layer	Technology	Purpose
Runtime	Node.js 22+	JavaScript/TypeScript runtime
Language	TypeScript (strict mode)	Type safety across the codebase
Package Manager	pnpm 10	Monorepo workspace management
API Framework	Fastify	High-performance HTTP server
Frontend	Next.js 15	React SSR/SSG framework
Database	PostgreSQL 16	Primary data store (source of truth)
ORM	Drizzle ORM	Type-safe database queries & migrations
Queue	BullMQ	Background job processing
Cache/Queue Backend	Redis 7	BullMQ backend, rate limiting
Validation	Zod	Runtime schema validation
Logging	Pino	Structured JSON logging
Testing	Vitest	Unit and integration tests
Linting	ESLint + Prettier	Code quality and formatting
Containerization	Docker Compose	Development environment

Why These Choices

Technology	Problem Solved	Alternative	Tradeoff
Fastify	High-performance API	Express, Hono	Faster than Express; smaller ecosystem
PostgreSQL	Reliable data store with FTS	MySQL, MongoDB	Best JSON + relational + FTS combo
Drizzle ORM	Type-safe SQL	Prisma, TypeORM	Closer to SQL; smaller bundle
BullMQ	Reliable job processing	Agenda, custom queues	Redis-backed; retries, backoff, observability
Redis	Queue backend, rate limiting	PostgreSQL LISTEN/NOTIFY	Fast; well-suited for queues
Next.js	SSR/SSG React frontend	Remix, Vite SPA	Server rendering for SEO; API routes
Zod	Runtime validation	Yup, Joi	TypeScript-first; composable schemas

4. Architecture Overview

System Architecture



	Redis	
	Port	
	6379	
+-----+		

Failure Modes

Failure	Effect
Postgres down	<code>/ready</code> returns 503; <code>/health</code> still 200; public reads fail
Redis down	<code>/ready</code> returns 503; background jobs pause; public reads still work
Worker crash	API and web still run; no new ingest/analysis
API crash	Web homepage shows "unreachable"; worker still processes
LLM unavailable	Analysis jobs fail; existing published data unaffected

Key Design Decisions

- **Modular Monolith first:** One process per service, clear module boundaries, no premature microservices
- **Evidence-first:** Claims backed by sources, not just LLM output
- **Human-in-the-loop:** AI generates drafts; humans review and publish
- **Append-only history:** Impact assessments are versioned; corrections create new versions
- **Separate evidence_strength from analysis_confidence:** Never collapse into one number

5. Repository Structure

```

bharatlens/
  apps/
    api/                                # Fastify HTTP API
      src/
        modules/
          admin/                        # Auth, review routes
          analytics/                    # Analytics routes
          countries/                    # Country routes
          events/                       # Event routes (CRUD, detail, sources, claims, impact, chain,
          map/                           # Geopolitical map routes
          topics/                        # Topic routes
        plugins/
          error-handler.ts             # Global error handling
          public-api.ts                 # CORS, rate limiting
          request-id.ts                 # Request ID generation
        infrastructure/
          redis.ts                      # Redis connection
          server.ts                     # Entry point
          app.ts                        # Fastify app setup
          types.ts                      # TypeScript declarations
      worker/                          # BullMQ background worker
        src/
          processors/
            ingest.ts                   # RSS feed ingestion processor
            claims.ts                   # Claims extraction processor
            analysis.ts                  # AI analysis processor
          infrastructure/
            redis.ts                    # Redis connection
            scheduler.ts                 # Cron-like RSS polling
            server.ts                    # Entry point
            app.ts                       # Worker setup
      web/                             # Next.js frontend
        app/                           # App Router pages
          page.tsx                      # Homepage
          events/                       # Event listing and detail
          countries/                     # Country pages
          topics/                        # Topic pages
          map/                           # Geopolitical map
          analytics/                     # Analytics dashboard
          search/                        # Full-text search
          about/                         # About page
          admin/                         # Admin console
        components/                     # React components
        lib/                            # API client, utilities
        styles/                         # Global CSS

```

```

packages/
  shared/                # Domain enums, lifecycle, validation
    src/
      domain.ts          # EventStatus, ClaimType, ImpactLevel, etc.
      lifecycle.ts       # Status transition rules
      errors.ts          # Typed domain errors
      evidence.ts        # Evidence strength computation
      timeline.ts        # Timeline helpers
      chain.ts           # Impact chain graph validation

  config/                # Zod-validated environment config
  database/              # Drizzle schema, migrations, queries
    src/
      schema/            # 12+ schema files (events, articles, claims, etc.)
      migrate.ts         # Migration runner
      seed.ts            # Seed data
      catalog.ts         # EventCatalog write API
      queries.ts         # Read queries
      client.ts          # Database connection

  jobs/                  # Queue names, payloads, enqueue helpers
    src/
      queues.ts          # Queue definitions
      payloads.ts        # Job payload schemas
      job-ids.ts         # Deterministic job IDs
      enqueue.ts         # Enqueue helpers
      cli.ts             # Queue CLI

  ingestion/             # RSS adapter, dedup, claims extraction
    src/
      adapter.ts         # NewsSourceAdapter interface
      rss-adapter.ts     # RSS/Atom implementation
      feeds.ts           # Feed configurations
      normalize.ts       # URL/title normalization
      dedupe.ts          # Multi-layer deduplication
      pipeline.ts        # Full ingestion pipeline
      claims.ts          # Claims extraction logic
      candidates.ts      # Candidate event discovery

  ai/                    # Structured analysis pipeline
    src/
      schema.ts          # Zod output schema
      prompt.ts          # Versioned prompts
      provider.ts        # LLM provider (OpenAI-compatible)
      pipeline.ts        # Full analysis pipeline
      sanitize.ts        # Output sanitization
      cli.ts             # CLI runner
      eval/              # Evaluation harness

  logging/               # Pino logger with secret redaction

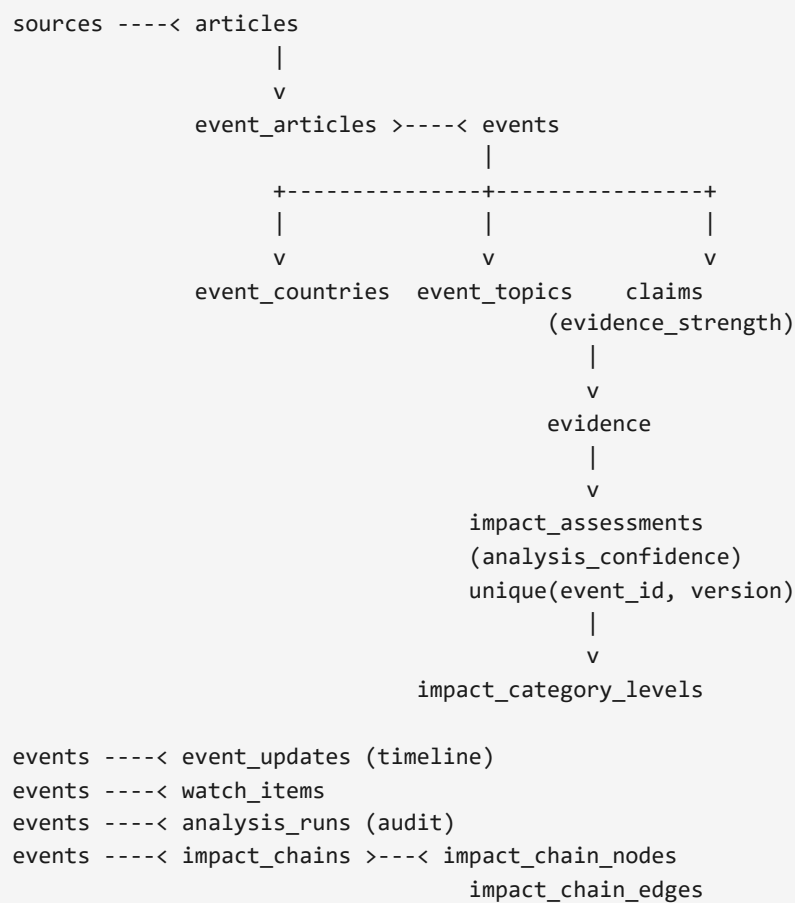
```

Package Dependencies

```
@bharatlens/shared      (no deps - pure domain)
  |
  v
@bharatlens/config      (depends on shared)
  |
  v
@bharatlens/logging     (depends on config)
  |
  v
@bharatlens/database    (depends on config, shared)
  |
  +---> @bharatlens/jobs      (depends on database, shared)
  +---> @bharatlens/ingestion (depends on database, shared)
  +---> @bharatlens/ai        (depends on database, shared, config)
  |
  v
apps/api                (depends on database, shared, config, logging, jobs)
apps/worker              (depends on database, shared, config, logging, jobs, ai, ingestion)
apps/web                 (depends on shared, config)
```

6. Database Schema & Domain Model

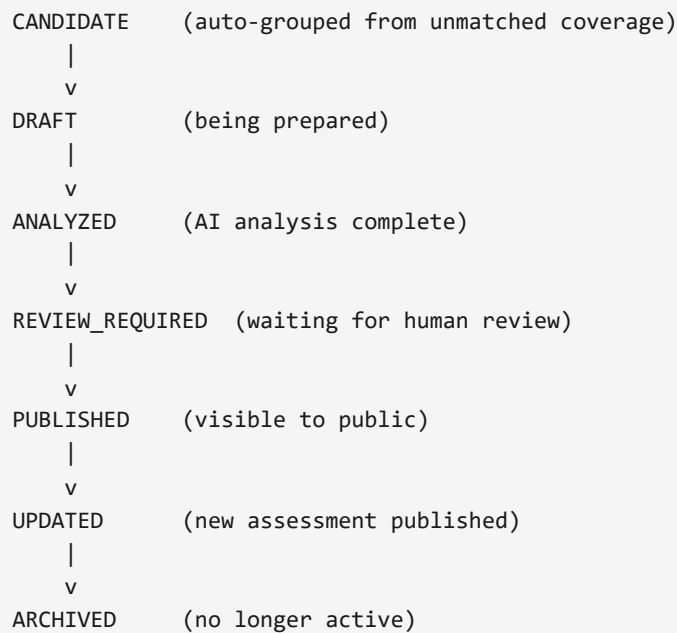
Entity Relationship Overview



Core Tables

Table	Purpose	Key Fields
events	One table for all events (candidate via status)	id, slug, title, summary, status, importance, occurred_at, current_impact_assessment_id
articles	Ingested article metadata (not full text)	id, source_id, url, title, content_hash, status, duplicate_of_article_id
sources	Publishers (Reuters, BBC, MEA, etc.)	id, name, slug, type (NEWS GOVERNMENT INTERNATIONAL_ORG)
claims	Structured statements on events	id, event_id, statement, type (FACT/ANALYSIS/SCENARIO/UNKNOWN), status, evidence_strength
evidence	Short excerpts + URLs supporting claims	id, claim_id, source_id, url, excerpt
impact_assessments	Versioned India Impact assessments	id, event_id, version, overall_level, analysis_confidence, reasoning
impact_category_levels	Per-category impact levels	assessment_id, category, level
event_updates	Timeline rows	id, event_id, title, description, update_time
impact_chains	Versioned causal pathways	id, event_id, version, status
impact_chain_nodes	Chain nodes (ROOT/CHANNEL/IMPACT)	chain_id, key, kind, label, description
impact_chain_edges	Chain edges (directed graph)	chain_id, from_key, to_key
analysis_runs	AI analysis audit trail	id, event_id, model_name, prompt_version, output, status
ingestion_jobs	Ingestion run audit	id, source_slug, items_seen, items_inserted, status
countries	World countries	code (ISO-2), name, slug
topics	Event topic categories	slug, name

Event Status Lifecycle



Key Constraints

- `events.slug` unique - public URLs
- `articles.url` unique - ingest URL deduplication
- `articles (source_id, external_id)` unique - RSS GUID per source
- `claims (event_id, statement)` unique - idempotent extraction
- `evidence (claim_id, url)` unique - one URL per claim
- `impact_assessments (event_id, version)` unique - append-only versions
- Circular FK: assessments reference events; events reference the current assessment (`ON DELETE SET NULL`)

Important Indexes

Index	Why
<code>events_status_idx</code>	Admin vs public listings
<code>events_occurred_at_idx</code>	Chronological pages
<code>events_published_at_idx</code>	Recent events
<code>articles_content_hash_idx</code>	Dedupe lookups
<code>articles_normalized_title_idx</code>	Title-based dedup
<code>claims_event_id_idx</code>	Event detail page
<code>evidence_claim_id_idx</code>	Evidence panel
<code>impact_assessments_event_id_idx</code>	Impact history

7. API Reference

Response Envelope

```
// Success
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "limit": 20, "total": 42, "pageCount": 3 }
}

// Error
{
  "success": false,
  "error": {
    "code": "EVENT_NOT_FOUND",
    "message": "Event not found"
  }
}
```

Public Endpoints

Method	Path	Description
GET	/health	Liveness check
GET	/ready	Readiness (Postgres + Redis)
GET	/api/v1/events	List events (paginated, filterable)
GET	/api/v1/events/:slug	Event detail
GET	/api/v1/events/:id/sources	Event sources
GET	/api/v1/events/:id/claims	Event claims + evidence
GET	/api/v1/events/:id/impact	Current + history assessments
GET	/api/v1/events/:id/chain	Impact chain graph
GET	/api/v1/events/:id/updates	Event timeline
GET	/api/v1/countries	List countries
GET	/api/v1/countries/:code	Country detail
GET	/api/v1/topics	List topics
GET	/api/v1/topics/:slug	Topic detail
GET	/api/v1/search?q=...	Full-text search
GET	/api/v1/map/overview	Map data (country partners)
GET	/api/v1/analytics/overview	Analytics data

Events List Query Parameters

Param	Values	Default
page	1+	1
limit	1-100	20
country	ISO-2 code	-
topic	Topic slug	-
importance	LOW, MEDIUM, HIGH, CRITICAL	-
sort	occurredAt, publishedAt, updatedAt	occurredAt
order	asc, desc	desc

Admin Endpoints

Method	Path	Description
POST	/api/v1/admin/auth/login	Login (password, rate limited 10/min)
POST	/api/v1/admin/auth/logout	Clear session cookie
GET	/api/v1/admin/overview	Dashboard stats
GET	/api/v1/admin/events	List all events (any status)
GET	/api/v1/admin/events/:slug	Full detail incl. drafts
POST	/api/v1/admin/events	Create candidate event
PATCH	/api/v1/admin/events/:id	Edit event metadata
POST	/api/v1/admin/claims/:id/review	Approve/reject claim
PUT	/api/v1/admin/assessments/:id	Edit draft assessment
POST	/api/v1/admin/assessments/:id/publish	Publish assessment
POST	/api/v1/admin/events/:id/analyze	Enqueue analysis job
PUT	/api/v1/admin/chains/:id	Edit draft chain
POST	/api/v1/admin/chains/:id/publish	Publish chain
POST	/api/v1/admin/feeds/:slug/ingest	Enqueue feed ingestion

8. Data Ingestion Pipeline

Pipeline Flow

```
Feed Config (bbc-world, un-news)
  |
  v
Fetch RSS/Atom (HTTP with timeout + User-Agent)
  |
  v
Parse Items (title, link, guid, date, description)
  |
  v
Normalize (canonical URL, collapsed title, stripped HTML)
  |
  v
SHA-256 Content Hash (title + day + summary)
  |
  v
Deduplication
  |-- Hard: unique URL / (source, external_id)
  |-- Soft: content hash, exact title, token overlap
  |
  v
Insert Article (status: NORMALIZED)
  |
  v
Record ingestion_jobs row
  |
  v
Candidate Event Discovery (if new articles cluster)
```

What Is Stored

BharatLens does NOT scrape full articles. It stores only metadata:

- Source metadata (publisher name, URL)
- Article title
- Canonical URL
- Optional short summary (max 500 characters)
- Publication time
- Content hash (SHA-256)

News Source Adapter

```
interface NewsSourceAdapter {  
  fetchArticles(): Promise<RawArticle[]>;  
}
```

Currently ships one adapter: `RssAdapter`. A future News API adapter can implement the same interface.

Current Feeds

Slug	Source
<code>bbc-world</code>	BBC World News
<code>un-news</code>	UN News

9. Deduplication Strategy

Multi-Layer Approach

Layer	Method	Match Type	Window
1	URL	Hard (no second row)	None
2	External ID (RSS GUID)	Hard (no second row)	None
3	Content hash	Soft (status=DUPLICATE)	None
4	Exact normalized title	Soft (status=DUPLICATE)	48 hours
5	Token overlap	Soft (status=DUPLICATE)	48 hours

Why Not Semantic Embeddings?

- Volume is tiny - PostgreSQL unique constraints are sufficient
- Embeddings require a vector database (premature)
- False positives from embeddings are harder to debug
- Token overlap + gazetteer entity is precise enough for this scale
- False negatives are accepted until evidence shows they matter

Candidate Event Discovery

After ingest inserts new articles, the worker scans unmatched coverage and clusters it with the precision dedupe matcher inside a 48-hour window. Clusters that pass three gates become CANDIDATE events:

1. At least 2 articles
2. A shared gazetteer entity (country, organization name)
3. A deterministic India-relevance token

10. Claims & Evidence System

Claim Types

Type	Meaning	Source
FACT	Directly supported by reliable evidence	Human reviewer only
ANALYSIS	Reasoned interpretation based on facts	Extractor or model
SCENARIO	A possible future outcome	Extractor or model
UNKNOWN	Insufficient reliable information	Extractor or model

Key Rule: RSS headline extraction and model-generated claims are NEVER typed FACT. FACT stays a reviewer/seed decision only.

Evidence Strength Computation

`evidence_strength` is computed from linked `evidence` rows:

- Count distinct `source_id` values -> `sourceCount` / `independentSourceCount`
- Count distinct sources with type GOVERNMENT or INTERNATIONAL_ORG -> `officialSourceCount`
- Classification: WEAK (1 source) / MODERATE (2-3) / STRONG (4+)

An LLM percentage is never stored. `evidenceReason` restates the counts in natural language.

Claims Extractor (Deterministic)

```
Published event
|
v
Find articles sharing gazetteer entity + 2 tokens with event
|
v
Attach article to event (event_articles)
|
v
If article overlaps existing claim -> add evidence URL + excerpt
Else -> PENDING claim (UNKNOWN/ANALYSIS/SCENARIO) + evidence
```

Re-runs are idempotent via unique constraints on `(event, statement)` and `(claim, url)`.

11. AI Analysis Pipeline

Pipeline Flow

```
Event metadata + linked claims/articles
|
v
Versioned prompt (event-analysis-v1)
|
v
Model call (HTTP if LLM_API_KEY set, otherwise deterministic stub)
|
v
JSON parse -> Zod schema validation -> URL allowlist check
|
v
Sanitization (strip extra fields, validate enums)
|
v
analysis_runs row (audit) + DRAFT impact assessment
|
v
Human review (M10) -> Publish
```

What Is Stored

Field	Purpose
<code>analysis_runs.model_name</code>	Which model produced the JSON
<code>analysis_runs.prompt_version</code>	<code>event-analysis-v1</code> today
<code>analysis_runs.input_references</code>	Event, claim IDs, article IDs, allowed URLs
<code>analysis_runs.output</code>	Validated JSON after sanitization
<code>impact_assessments.analysis_confidence</code>	LOW / MEDIUM / HIGH (estimate)
<code>impact_assessments.evidence_strength</code>	Computed from linked evidence (not the model)

Safety Checks

1. **Schema validation (Zod):** Extra fields stripped; wrong enums fail the run
2. **URL allowlist:** Any `http(s)` URL not in the input allowlist fails the run
3. **Claim type:** Model claims are stored `PENDING` and never as `FACT`
4. **Confidence:** `analysisConfidence` cannot be a percentage
5. **UNKNOWN is valid:** If evidence is thin, UNKNOWN / LOW confidence is correct

Failure Modes

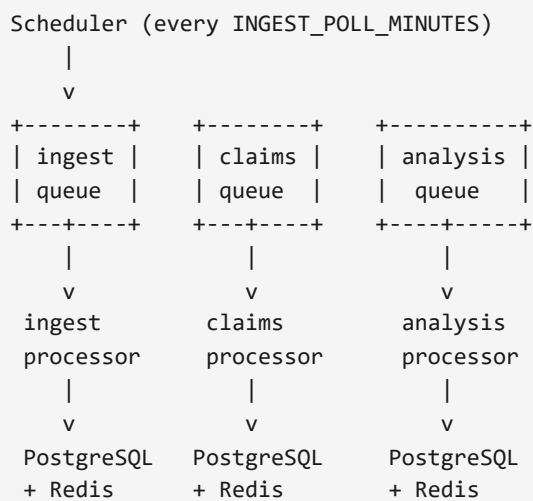
Failure	Effect
Invalid JSON / Zod failure	<code>analysis_runs</code> status = FAILED; no assessment created
Invented URL	FAILED; no assessment created
LLM HTTP error / timeout	FAILED; no assessment created
Re-run	New run + next DRAFT version (append-only)

Configuration

```
# Optional. Without it, uses deterministic stub.  
LLM_API_KEY=your-api-key  
LLM_MODEL=gpt-4o-mini  
LLM_BASE_URL=https://api.openai.com/v1
```

12. Background Processing (BullMQ)

Queue Architecture



Job Properties

Property	Value
Retries	5 attempts
Backoff	Exponential from 5 seconds
Lock duration	120 seconds (analysis)
Failed job retention	7 days in Redis
Payload format	Slugs (processors reload from DB)

Idempotency Strategy

- **Ingest:** Article-level dedup (unique URL, external ID)
- **Claims:** Unique constraints (event_id, statement)

- **Analysis:** Per-day deterministic job IDs + DRAFT-assessment guard

Manual Queue Operations

```
# Enqueue feed ingestion
pnpm queue ingest --feed=bbc-world

# Enqueue claims refresh
pnpm queue claims --event=strait-of-hormuz-shipping-disruption

# Enqueue analysis (skips if draft exists today)
pnpm queue analyze --event=strait-of-hormuz-shipping-disruption

# Force analysis (always runs)
pnpm queue analyze --event=strait-of-hormuz-shipping-disruption --force
```

Worker Endpoints

Endpoint	Purpose
GET /health	Liveness + queue names
GET /ready	Postgres + Redis checks
GET /queues	Per-queue depth (waiting/active/completed/failed)

13. Admin Console & Human Review

Review Workflow

```
CANDIDATE / REVIEW_REQUIRED event
|
v
Review Workspace (/admin/events/[slug])
|
+---> Inspect claims + evidence
|      --> APPROVED / REJECTED (public sees APPROVED only)
|
+---> Run AI analysis
|      --> Enqueues analysis job (worker executes)
|
+---> Edit DRAFT assessment
|      --> Levels, reasoning, categories
|
+---> Publish
|      --> Transaction: DRAFT -> PUBLISHED
|      + lifecycle walk to public status
|      + current_impact_assessment_id swap
```

Admin Pages

Page	Purpose
<code>/admin/login</code>	Password authentication
<code>/admin</code>	Dashboard: status counts, queue depth, pipeline health
<code>/admin/events</code>	Event list with status filters
<code>/admin/events/[slug]</code>	Review workspace: claims, assessments, chains, analysis runs

Authentication

- HMAC-signed httpOnly session cookie
- Password from `ADMIN_PASSWORD` env var

- Rate limited to 10 login attempts per minute
- Session TTL configurable via `ADMIN_SESSION_TTL_HOURS`
- API refuses to boot without `ADMIN_PASSWORD`

Key Invariants

- Published versions are immutable
- Corrections create and publish new versions
- Publishing refuses assessments without categories
- Re-publishing already-published versions is rejected
- Draft assessments are not visible on public API

14. Impact Chains

What Is an Impact Chain?

An impact chain is a causal pathway showing how a global event leads to India-facing consequences:

```
Hormuz disruption (ROOT)
  |
  v
Shipping disruption (CHANNEL)
  |
  v
Oil transportation risk (CHANNEL)
  |
  v
Higher energy costs (CHANNEL)
  |
  v
India's import bill (IMPACT)
  |
  v
Inflation pressure (IMPACT)
  |
  v
Rupee / monetary policy (IMPACT)
```

Data Model

Table	Purpose
<code>impact_chains</code>	Versioned chain container (v1, v2, ...)
<code>impact_chain_nodes</code>	Nodes with key, kind (ROOT/CHANNEL/IMPACT), label, description
<code>impact_chain_edges</code>	Directed edges (from_key -> to_key)

Graph Validity Rules

- Exactly one ROOT node
- No cycles (acyclic graph)

- All nodes reachable from root
- Enforced by `validateChainGraph` in `@bharatlens/shared`

Lifecycle

1. AI proposes a DRAFT chain (prompt v2)
2. Reviewer edits in admin console
3. Reviewer publishes (freezes version)
4. `events.current_impact_chain_id` updated
5. Published versions are immutable history

Rendering

The public event page renders the current chain as a top-down stepper, layered client-side from the graph using a longest-path DAG layering algorithm. No per-event hardcoding.

15. Frontend Pages

Public Pages

Page	Route	Description
Homepage	/	Hero, high-impact events, trending topics, countries to watch
Event Listing	/events	Filterable grid with country/topic/importance filters
Event Detail	/events/[slug]	Full event: what happened, why, India impact, chain, timeline, claims, sources
Countries	/countries	Country directory grid
Country Detail	/countries/[code]	Country's events with pagination
Topic Detail	/topics/[slug]	Topic's events with pagination
Map	/map	India-centered orthographic globe with choropleth
Analytics	/analytics	Exposure grid, trend chart, topic/country movers
Search	/search	Full-text search with PostgreSQL FTS
About	/about	Product explanation

Admin Pages

Page	Route	Description
Login	/admin/login	Password authentication form
Dashboard	/admin	Status counts, queue depth, pipeline health, recent analysis runs
Event List	/admin/events	All events with status filters
Review	/admin/events/[slug]	Claims review, draft assessment editor, chain editor, analysis runs

Event Detail Page Structure

```
Event Detail (/events/[slug])
|
+---> Title + Summary
+---> "What happened?" (description)
+---> "Why did it happen?" (context)
+---> "Why does India care?" (impact reasoning)
+---> India Impact Panel
|   +---> Overall Level (HIGH/MEDIUM/LOW)
|   +---> Category levels (Energy, Trade, Security...)
|   +---> Evidence Strength
|   +---> Analysis Confidence
+---> Impact Chain (graph visualization)
+---> Timeline (event updates)
+---> Impact History (versioned assessments)
+---> Claims + Evidence
|   +---> FACT / ANALYSIS / SCENARIO labels
|   +---> Source attribution
+---> "What to Watch Next"
+---> Sources
```

Key Components

Component	Purpose
event-card.tsx	Event summary card for listings
impact-badge.tsx	Impact level indicator
impact-history.tsx	Version history visualization
event-timeline.tsx	Timeline visualization
impact-chain-flow.tsx	Graph chain visualization
review-actions.tsx	Admin review buttons and editors

16. Search System

Implementation

Search uses PostgreSQL full-text search with a stored generated tsvector:

- `events.search_vector` - generated column, english configuration
- Weighted: A=title, B=summary, C=description
- Queries use `websearch_to_tsquery` matching
- ILIKE fallback for partial words
- Results ranked by `ts_rank` when `sort=relevance`

Search Endpoint

```
GET /api/v1/search?q=india+oil&sort=relevance
```

Why Not Elasticsearch?

- Volume is tiny (hundreds of events, not millions)
- PostgreSQL FTS is sufficient for this scale
- Adding Elasticsearch introduces operational complexity
- Can evaluate later when actual requirements justify it

17. Analytics & Map

Analytics (/analytics)

All analytics are **derived at read time** from published/audit tables. No counters, no averages of ordinal levels.

Analytics Components

Component	Description
Category Exposure Grid	Distribution of impact levels across categories for current published assessments
Publication Trend	Weekly publication count + version upgrade/downgrade transitions
Topic Movers	Topics with biggest change vs prior equal-length window
Country Movers	Countries with biggest change vs prior equal-length window

Window defaults to 90 days, clamped to 7-365.

Geopolitical Map (/map)

- India-centered orthographic globe
- Built with d3-geo + topojson-client (no wrapper library)
- Choropleth intensity derived from shared-event counts
- Clicking a partner shows shared published events
- No curated or subjective relationship data

18. Development Setup

Prerequisites

- Node.js 22+
- pnpm 10 (via Corepack: `corepack enable`)
- Docker Desktop (for PostgreSQL and Redis)

Quick Start (Docker)

```
cp .env.example .env
docker compose up -d postgres redis
pnpm install
pnpm --filter @bharatlens/shared build
pnpm --filter @bharatlens/config build
pnpm --filter @bharatlens/logging build
pnpm --filter @bharatlens/database build
pnpm db:migrate
pnpm db:seed
pnpm ingest:rss
docker compose up --build
```

Local Development

```
docker compose up postgres redis
cp .env.example .env
pnpm install
pnpm --filter @bharatlens/shared build
pnpm --filter @bharatlens/config build
pnpm --filter @bharatlens/logging build
pnpm --filter @bharatlens/database build
pnpm db:migrate
pnpm db:seed
pnpm dev:worker
pnpm dev:api
pnpm dev:web
```

Environment Variables

Variable	Required	Default	Description
<code>NODE_ENV</code>	No	development	Environment mode
<code>LOG_LEVEL</code>	No	info	Log verbosity
<code>API_PORT</code>	No	3001	API listen port
<code>DATABASE_URL</code>	Yes	-	PostgreSQL connection string
<code>REDIS_URL</code>	Yes	-	Redis connection string
<code>ADMIN_PASSWORD</code>	Yes (API)	-	Admin login password (min 8 chars)
<code>INGEST_POLL_MINUTES</code>	No	30	RSS polling interval (0 = off)
<code>LLM_API_KEY</code>	No	-	OpenAI-compatible API key
<code>LLM_MODEL</code>	No	gpt-4o-mini	Model name

Access Points

Service	URL
Frontend	http://localhost:3000
API	http://localhost:3001
Worker	http://localhost:3002
PostgreSQL	localhost:5433
Redis	localhost:6379

19. Key Commands Reference

Command	Purpose
<code>pnpm dev:api</code>	Start API server (dev mode with hot reload)
<code>pnpm dev:worker</code>	Start background worker (dev mode)
<code>pnpm dev:web</code>	Start Next.js frontend (dev mode)
<code>pnpm db:migrate</code>	Apply Drizzle SQL migrations
<code>pnpm db:seed</code>	Seed hand-authored published events
<code>pnpm db:generate</code>	Generate migration files from schema changes
<code>pnpm ingest:rss</code>	Pull RSS metadata into articles
<code>pnpm claims:extract</code>	Attach article evidence; create PENDING claims
<code>pnpm analyze:event</code>	Run AI analysis (creates DRAFT assessment)
<code>pnpm eval:ai</code>	Run AI evaluation harness
<code>pnpm queue ingest --feed=X</code>	Enqueue feed ingestion for worker
<code>pnpm queue claims --event=X</code>	Enqueue claims refresh for worker
<code>pnpm queue analyze --event=X</code>	Enqueue analysis for worker
<code>pnpm test</code>	Run unit tests (Vitest)
<code>pnpm typecheck</code>	Type-check all packages
<code>pnpm lint</code>	Run ESLint
<code>pnpm format</code>	Format with Prettier
<code>pnpm format:check</code>	Check formatting
<code>pnpm build</code>	Build all packages

20. Architecture Decision Records

ADR	Decision	Summary
ADR-001	Modular Monolith	Start with one process per service, clear module boundaries, no premature microservices
ADR-002	PostgreSQL	Primary data store; best combo of relational, JSON, and FTS for this scale
ADR-003	Redis	Queue backend and rate limiting; not used as cache in M0
ADR-004	BullMQ	Background job processing with retries, backoff, observability; introduced in M9
ADR-005	Human Review	AI generates drafts; humans review and publish; never auto-publish
ADR-006	Evidence-First	Claims backed by sources; separate evidence_strength from analysis_confidence
ADR-007	Admin Auth	HMAC-signed httpOnly cookie; password from env; rate limited
ADR-008	Impact Chains	Versioned relational graphs; validated for acyclicity; rendered as DAG
ADR-009	Shared World Data	Countries, topics as first-class entities; not hardcoded in frontend
ADR-010	Map Rendering	d3-geo + topojson-client; no wrapper library; India-centered orthographic
ADR-011	Analytics Derivation	Derived at read time; no counters; no averaged ordinal levels
ADR-012	Claims Review	Claims require human approval; PENDING -> APPROVED/REJECTED workflow

21. Failure Modes & Observability

Failure Modes

Component	Failure	Effect	Mitigation
PostgreSQL	Down	Public reads fail; <code>/ready</code> = 503	Connection pooling; retry logic
Redis	Down	Background jobs pause; public reads still work	Graceful degradation; queue jobs fail safely
Worker	Crash	No new ingest/analysis; API+web unaffected	Process manager restarts; jobs are retryable
API	Crash	Web shows "unreachable"; worker unaffected	Health checks; process restart
LLM	Unavailable	Analysis jobs fail; existing data unaffected	Deterministic stub fallback; no auto-publish
Feed HTTP	Error	Specific feed job fails; other feeds continue	Per-feed job isolation; 5 retries
Malformed LLM output	Zod validation fails	Run status = FAILED; no assessment created	Schema validation before storage

Observability Metrics

Metric	Source
API latency	Request logging with duration
API error rate	Error handler + request logging
Articles ingested	<code>ingestion_jobs</code> table
Duplicates detected	<code>ingestion_jobs.duplicate_reasons</code>
AI analyses generated	<code>analysis_runs</code> table
AI failures	<code>analysis_runs</code> status = FAILED
Job failures	BullMQ failed jobs + worker logging
Queue depth	<code>GET /queues</code> endpoint

Structured Logging

Every request includes: `requestId` , `method` , `path` , `statusCode` , `duration` .

Background jobs include: `jobId` , `jobType` , `attempt` , `duration` , `status` .

AI jobs include: `analysisId` , `model` , `promptVersion` , `eventId` , `duration` , `status` .

22. AI Evaluation Harness

Overview

The evaluation harness (M15) validates AI behavior against fixtures. It does NOT use model-emitted percentages as accuracy scores.

Two Tiers

1. Offline (runs in normal test suite)

- Adversarial output corpus pushed through parse -> validate -> sanitize
- Proves schema enforcement, FACT stripping, URL-fabrication rejection
- No network calls required

2. Live Scenarios

- 6 gold-labeled fixtures
- Scored on behavioral counts:
 - Schema pass rate
 - Raw FACT leaks
 - Fabrication caught
 - Chain validity
 - Overall-level exact vs adjacent-rank agreement
 - Dominant claim-type agreement
- Results compared against committed `baseline.json`
- `--check` flag fails on any dropped metric

Commands

```
# Run evaluation with deterministic stub
pnpm eval:ai --model=stub

# Run with live LLM
pnpm eval:ai --model=live

# Check against baseline (CI mode)
pnpm eval:ai --check

# Update baseline after intentional changes
pnpm eval:ai --update-baseline
```

Key Principles

- Levels are never merged into a score
- Model-emitted confidence is never treated as accuracy
- Baseline updates are deliberate commits alongside prompt/stub changes
- Fixtures are gold-labeled by human review

23. Design Principles

Simplicity First

- Prefer the simplest architecture that solves the current problem
- No premature microservices, Kafka, Elasticsearch, or Kubernetes
- PostgreSQL is sufficient for search at current scale
- Redis is used only when it solves a real problem

Evidence-First Architecture

- Never trust raw LLM output
- Claims backed by sources, not just model output
- Separate evidence_strength from analysis_confidence
- FACT claims require human review
- UNKNOWN is a valid result (preferred over hallucination)

Political Neutrality

- No promotion of political parties or ideologies
- No portrayal of countries as inherently good or bad
- No inflammatory language or nationalist propaganda
- Source transparency over manufactured confidence

Source Transparency

- Every important claim has supporting sources
- Never fabricate URLs, quotes, statistics, or sources
- Store metadata, not full copyrighted articles
- Respect robots.txt and API terms

AI Safety

- Never fabricate events, sources, quotes, or statistics
- Never present speculation as fact

- Never claim certainty without evidence
- Prefer "insufficient evidence" over hallucinating
- Every AI output is auditable (model, prompt version, sources, review status)

Append-Only History

- Impact assessments are versioned (v1, v2, v3...)
- Corrections create new versions, never overwrite
- Event updates form a timeline
- Analysis runs are an audit trail
- Published versions are immutable

Human-in-the-Loop

- AI generates drafts; never auto-publishes
- Humans review claims, assessments, and chains
- Admin console provides full inspection tools
- Public API only shows approved/published data

Fail Fast

- Missing required env vars crash on boot
- Invalid config is caught at startup
- Zod validates all AI output
- Unique constraints prevent duplicate data

BharatLens - See the world through India's lens.

Complete Project Guide | Version M15 | August 2026

Repository: C:\N-Nayan\Projects\BharatLens